

Reviewer Pack — Hilbert Series Complexity of Orbit Closures for Permanent vs...

Entry 4cd8cbfd5e94 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 14:45:39 UTC

Hilbert Series Complexity of Orbit Closures for Permanent vs Determinant

Entry ID: 4cd8cbfd5e94 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 14:45:39 UTC

1. Conjecture

Field A (mathematical branch): ALGEBRAIC_GEOMETRY_OF_ORBIT_CLOSURES **Field B** (complexity object): GEOMETRIC_COMPLEXITY_THEORY

Statement:

For all $n \geq 2$ and $m < n^{\{1.5\}}$, the Hilbert series of the coordinate ring of the orbit closure of the permanent polynomial perm_n under $GL(n)$ has a strictly higher degree than the Hilbert series of the orbit closure of the determinant polynomial det_m under $GL(m)$, and this invariant is preserved under linear substitutions.

Rationale (proposer's reasoning):

The Hilbert series encodes the growth of the dimensions of graded components in the coordinate ring of an orbit closure. If perm_n 's orbit closure has a higher-degree Hilbert series than det_m 's, it suggests structural complexity differences that could contribute to separating their complexity classes. This ties to GCT's goal of using algebraic geometry to distinguish det and perm .

Taxonomy category: GCT_DET_PERM (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 73b808c37ef3f825

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    augmented = [A[i] + [b[i]] for i in range(m)]
    for i in range(n):
        max_row = i
        for j in range(i+1, m):
            if abs(augmented[j][i]) > abs(augmented[max_row][i]):
                max_row = j
        augmented[i], augmented[max_row] = augmented[max_row], augmented[i]
        pivot = augmented[i][i]
        for j in range(n + 1):
            augmented[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = augmented[j][i]
                for k in range(n + 1):
```

```

        augmented[j][k] -= factor * augmented[i][k]
    return [row[-1] for row in augmented]

def determinant(A):
    n = len(A)
    if n == 2:
        return A[0][0] * A[1][1] - A[0][1] * A[1][0]
    det = 0
    sign = 1
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in A[1:]]
        det += sign * A[0][i] * determinant(submatrix)
        sign *= -1
    return det

def hilbert_series(poly, n):
    m = len(poly)
    h = [Fraction(1)] + [Fraction(0)] * (m-1)
    new_h = [Fraction(0)] * (m+n-1)
    for i in range(m):
        for j in range(n-i+1):
            new_h[j+i-1] += h[j] * poly[i]
    return new_h

def permanent(matrix):
    n = len(matrix)
    if n == 1:
        return matrix[0][0]
    det = 0
    sign = 1
    for i in range(n):
        submatrix = [row[:i] + row[i+1:] for row in matrix[1:]]
        det += sign * matrix[0][i] * permanent(submatrix)
        sign *= -1
    return det

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        m = random.randint(1, math.isqrt(n**1.5))
        perm_n = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
        det_m = [[random.randint(-10, 10) for _ in range(m)] for _ in range(m)]

        perm_series = hilbert_series(permanent(perm_n), n)
        det_series = hilbert_series(determinant(det_m), m)

        perm_degree = len([x for x in perm_series if x != Fraction(0)]) - 1
        det_degree = len([x for x in det_series if x != Fraction(0)]) - 1

        results.append({
            "metric_name": "Hilbert Series Degree",
            "metric_value": perm_degree,
            "instances_tested": 1,

```

```

        "conjecture_holds": perm_degree > det_degree,
        "counterexample": "" if perm_degree > det_degree else f"perm_n={perm_n}, det_m={det_m}"
    })

    return {
        "metric_name": "Hilbert Series Degree",
        "metric_value": sum(result["metric_value"] for result in results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": all(result["conjecture_holds"] for result in results),
        "counterexample": next((result["counterexample"] for result in results if not result["conjecture_holds"]), None)
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3**j for i in range(5) for j in range(5)]

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a3564bca.py", line 124, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a3564bca.py", line 93, in run_trial
    m = random.randint(1, math.isqrt(n**1.5))
              ~~~~~^
TypeError: 'float' object cannot be interpreted as an integer

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to type error in calculating m; insufficient data to assess conjecture |
 next: Fix integer conversion in test code to handle $n^{1.5}$ properly

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	42347
2	propose	ollama_remote	qwen3:8b	0	0	111865
3	preregistration	ollama_remote	qwen3:8b	0	0	23976
4	novelty	ollama_remote	qwen3:8b	0	0	20495
5	novelty	ollama_remote	qwen3:8b	0	0	18469
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17518
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12718
8	judge	ollama_remote	qwen3:8b	0	0	17731

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 265119 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/4cd8cbfd5e94.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4cd8cbfd5e94.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4cd8cbfd5e94.tar.gz` (if generated)